Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Regional Transportation District

RTD Denver EMU

Project # 7230

Door System

Neuron Gateway Application Software Integration Test Plan (SITP)

Contract Number		CDRL Number	
		Customer Ref. Number	-

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	James Kim Software Designer	James Kim 2011-10-20
Verified :	Martin Ouellet, Eng. Software Department Supervisor	Daniel Beaumont for Martin Ouellet 2011-10-20
Approved :	Aldo Di Rocco Project Manager	2011-10-20

Document No.	**201111460**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-19**

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-10-19	James Kim	Official release.
0.1	---	2011-10-18	James Kim	Implementing comments from internal inspection I-1870.
0	---	2011-10-11	James Kim	Original issue.

Preface

This Software Integration Test Plan complies with the requirements of the IEEE Std 829-1998, and the IEEE Std. 1558-2004.

LONWORKS® and LONMARK® are registered trademarks of Echelon Corporation.

Table of Contents

Revision List 2
Preface 3
Table of Contents 4
List of Figures 6
List of Tables 6

1 TEST PLAN SPECIFICATION IDENTIFIER 7

2 INTRODUCTION 7

2.1 Objective 7
2.2 Scope 7
2.3 Definitions and Acronyms 7
2.3.1 Definitions 7
2.3.2 Acronyms 7
2.4 References 9

3 TEST PLAN 10

3.1 Test Items 10
3.2 Features to be Tested 10
3.3 Features not Tested 10
3.4 Approach 10
3.4.1 Network Validation Tool 10
3.4.2 SPI Validation Tool 11
3.4.3 The Location ID Unit 11
3.5 Item Pass/Fail Criteria 11
3.6 Suspension Criteria and Resumption Requirements 11
3.6.1 Suspension Criteria 11
3.6.2 Resumption Requirements 11
3.7 Test Deliverables 12
3.8 Testing Tasks 12
3.9 Environmental Needs 12
3.9.1 Hardware 12
3.9.2 Software 12
3.10 Responsibilities 12
3.11 Staffing and Training Needs 12
3.12 Schedule 12
3.13 Risks and Contingencies 12
3.14 Approvals 12

4 TEST DESIGN 13

4.1 Test Identification 13
4.1.1 Neuron Node Self-Installation 13
4.1.2 Self-Test 13
4.1.3 Time Update Function 14
4.1.4 DCU Status 15
4.1.5 Network Health Monitoring 15
4.1.6 Version Management 15
4.1.7 DCU Communication via SPI Interface 16
4.1.8 SCI Update 19
4.1.9 LonMark® Interoperability 19

5 TEST PROCEDURE 20

5.1 Special Requirements 20
5.2 Procedure Steps 20

5.2.1 Test Log 20
5.2.2 Test Setup 21
5.3 Procedure 25
5.3.1 Neuron Node Self-Installation 25
5.3.2 Self-Test 26
5.3.3 Time Update with Network 27
5.3.4 DCU Status 28
5.3.5 Network Health Monitoring 28
5.3.6 Version Management 29
5.3.7 SCI Update 30
5.3.8 DCU Communication via SPI interface 31
5.3.9 LonMark® Interoperability 32

APPENDIX A: TEST REPORT SHEET 33

List of Figures

Figure 5-1: Test Setup 21
Figure 5-2: NodeUtil Main Menu 23
Figure 5-3: NodeUtil Device Menu 23
Figure 5-4: Electrical Connections of the Location ID Switches 24

List of Tables

Table 3-1: Item to Be Tested 10
Table 4-1: Location vs. Location Id Strings 13
Table 4-2: Version Field, Valid Test Vectors 18
Table 4-3: Version Field, Invalid Test Vectors 18
Table 5-1: Hardware Required 20
Table 5-2: Software Required 20
Table 5-3: Location ID and IDENT Switches 24

1 TEST PLAN SPECIFICATION IDENTIFIER

201111460 Revision 1, 2011-10-19

2 INTRODUCTION

This document is the Neuron Gateway Application **TC3 Software Integration Test Plan** (SITP) developed by Vapor Stone Rail Systems.

This document includes 3 major contents as stated in Software Quality Assurance Plan (SQAP) P/N 201111208, Section 4.9:

- Section 3, Software Integration Test Planning
- Section 4, Software Integration Test Design Specification
- Section 5, Software Integration Test Procedures Specification

2.1 Objective

The objective of this SITP is to specify the tests required to validate the Neuron Gateway Application.

2.2 Scope

The scope of this SITP applies to the software requirements of Neuron Gateway Application (NGA). The NGA interfaces between the DCU Application and the car network.

The intended audience is the Vapor Stone Rail Systems software development and maintenance team members of the Hyundai Rotem and the Denver Regional Transportation District.

2.3 Definitions and Acronyms

2.3.1 Definitions

LonWorks: Registered trademark of Echelon corp., which presents Echelon distributed network. See http://www.echelon.com

Event: Term used to globally identify an Info, an Exception or a Failure.

Info: Info is an occurrence of an Informative condition monitored by the system, which is neither an exception nor a failure.

Exception: An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).

Failure: A failure is an occurrence of a condition that is outside of the specified parameters and affecting this LRU. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

2.3.2 Acronyms

Acronym	Meaning
ASCII	American Standard Code for Information Interchange
CFG	Configuration
CPLD	Complex Programmable Logic Device
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DCU	Door Control Unit
FTP	File Transfer Protocol
GUI	Graphical User Interface

Acronym	Meaning
I/O	Input/Output
ID	Identification
IEEE	Institute of Electrical and Electronics Engineers, Inc.
LED	Light-Emitting Diode
LRU	Line Replace Unit
MDS	Monitoring and Diagnostic System
NA	Not Available
NGA	Neuron Gateway Application
NV	Network Variable
NVT	Network Validation Tool
OpenLDV	Echelon's application program interface for the network
P/N	Part Number
P/F	Pass or Fail
PC	Personal Computer
PCLTA	Echelon's PC card for network interface
PTU	Portable Test Unit
RTD	Regional Transportation District
SCI	Software Configuration Item
SITP	Software Integration Test Plan
SPI	Serial Peripheral Interface
SPR	Software Problem Report
SQAP	Software Quality Assurance Plan
SRS	Software Requirements Specification
Std	Standard
TL	Train Line
TS	Technical Specifications
VSRS	Vapor Stone Rail Systems

2.4 References

Standards

IEEE Std 829-1998	IEEE Standard for Software Test Documentation, 16 September 1998, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

201111185	System Functional Description, RTD Denver EMU Door System, Rev. 3
201111186	Software Configuration Management Plan, Door System, Rev. 1
201111207	Software Project Management Plan, Door System, Rev. 0
201111208	Software Quality Assurance Plan, Door System, Rev. 1
201111254	Software Verification and Validation Plan, Door System, Rev. 0
201111256	Software Requirement Specification, Door System – Neuron Gateway Application, Rev. 1
201111295	Software Design Description, Door System – Neuron Gateway Application, Rev. 0
201111586	Interface Control Document - Serial Peripheral Interface, Rev. 1
S17-10	Software Integration Test Procedure Template, Rev.2, 2002-01-15
S17-08	Software Integration Test Plan Template, Rev. 2, 2002-01-15

Quester Tangent Documents

DDN-0432-0080	MDS Car Monitoring Network Interface Control Document, Rev. 01
DDN-0432-0090	Door System Car Monitoring Network Interface Control Document, Rev. 0 – Draft

3 TEST PLAN

3.1 Test Items

All the items that make up the Neuron Gateway Application will be tested during the integration test. The versions tested will be placed under configuration management as defined in the Software Configuration Management Plan document P/N 201111186, Section 3.

The Software Requirements Specifications (SRS) of Neuron Gateway Application P/N 201111256 will provide the basis for defining the correct operation.

Table 3-1 lists the item to be tested.

Table 3-1: Item to Be Tested

Item	Software #
Neuron Gateway Application	2011L0052

3.2 Features to be Tested

The following lists the features that will be tested:

- Initialization Function
- Node Object Function
- Time Update Function
- File Transfer Protocol Function
- Software Version and Compile Time
- DCU Status Function
- SCI Update Function
- SPI Media Access Function
- Network Health Monitoring Function
- Location Identification Function
- Self Test Function
- LONWORKS® Media Access Function
- LONMARK® Interoperability Function

3.3 Features not Tested

None.

3.4 Approach

The following tools will be developed and used in the test procedure:

- The *Network Validation Tool* software simulates the Car Network, explores the Neuron node, and communicates with the *SPI Validation Tool*.
- The *SPI Validation Tool* exchanges data with the Neuron Gateway Application over the SPI interface, under the control of the *Network Validation Tool*.
- The *Location ID Unit* is a custom-made test jig providing manual stimulus onto the Location ID inputs of the Neuron Gateway Application.

3.4.1 Network Validation Tool

The *Network Validation Tool* is a Graphical-User-Interface (GUI) based Windows software that simulates the car network and SPI operation. It runs in console mode under Windows XP. The software has the following interfaces:

- GUI-based user interface;
- Echelon PCLTA -10 or PCLTA-20 network interface card;
- Serial Interface (RS232 COM Port) to the *SPI Validation Tool*.

The *Network Validation Tool* is able to test the Neuron Gateway Application through the following features:

- Retrieve Neuron ID, Configuration Structures, Domain Table, Address Table
- Control of the Node Object
- Simulate network Time Update (sent every hour)
- Update of the SCI (FTP)
- Simulate Software Version request
- Monitor for Status Information changes and record a histogram of the period at which the messages are received.
- Simulates periodic Health Checks request/Status Information response

The *Network Validation Tool* also communicates with the *SPI Validation Tool* to monitor the SPI data exchanged between Neuron Application and the *SPI Validation Tool*, adding the following features:

- Communication with the *SPI Validation Tool* via RS232
- Facilitation of change of the data sent over the SPI bus, including the Extra Data.
- Facilitation of rotation of the Extra Data.
- Monitoring and display of the received data over the SPI bus.

3.4.2 SPI Validation Tool

The *SPI Validation Tool* is a test software running on the DCU's main CPU. It has the following features:

- Accept SPI data frames from the Neuron Gateway Application, measure the period between the receptions of each SPI frame, and sends the frame and the measurement to the *Network Validation Tool* via RS232.
- Accepts data frame from the *Network Validation Tool*.
- Synchronously responds to the Neuron Gateway Application frames using the last frame received from the *Network Validation Tool*.

3.4.3 The Location ID Unit

The Location ID Unit is a box that has two switches, and it connects to the DCU.

3.5 Item Pass/Fail Criteria

The Neuron Gateway Application is to be considered acceptable if:

- It meets all the criteria defined in the Software Requirements Specifications (SRS) –Neuron Gateway Application or upon approval of non-conformities detected during the test by the customer.
- After undergoing all the tests, the Neuron Gateway Application remains functional without corruption of settings.

3.6 Suspension Criteria and Resumption Requirements

3.6.1 Suspension Criteria

The following criteria will cause a suspension of all testing activities:

- Inability to meet a criterion in the SRS;
- Non-conformities detected during the test are not approved by the customer.

3.6.2 Resumption Requirements

When a new version of the Neuron Gateway Application is transmitted to the test personnel after a suspension of testing has occurred, the test will be started over again.

3.7 Test Deliverables

The following documents will be generated by the test personnel and will be placed under configuration management after test completion:

- Neuron Gateway Application (NGA) Software Integration Test Plan (SITP)
- Neuron Gateway Application (NGA) Software Integration Test Report (SITR)

3.8 Testing Tasks

The testing tasks are defined in the SVVP Section 5.

3.9 Environmental Needs

3.9.1 Hardware

The software to be tested will run on a DCU, and a PC running Windows will connect to the DCU to provide test stimulus. The hardware used for the test is listed in the Table 5-1.

3.9.2 Software

Software loaded in DCU will be the latest version of the Neuron Gateway Application. The software used for the test is listed in the Table 5-2.

3.10 Responsibilities

The responsibilities are defined in the SVVP Section 5.5.

3.11 Staffing and Training Needs

The staffing needs are defined in the SVVP Section 5.4. No training is needed for this project.

3.12 Schedule

The schedule is defined in the SVVP Section 5.2.

3.13 Risks and Contingencies

The risks and contingencies are defined in the SQAP Section 14.

3.14 Approvals

Approvals names and signatures are included on the cover page of this document.

4 TEST DESIGN

4.1 Test Identification

4.1.1 Neuron Node Self-Installation

The following tests are performed with no SPI communication.

Table 4-1: Location vs. Location Id Strings

Location	Location Id String
DCU R1	DCU01
DCU R2	DCU02
DCU L1	DCU03
DCU L2	DCU04

4.1.1.1 Location DCU01

1) Select Location ID, and reboot software, then test that software responds to the right subnet/node address.
2) Change the Location ID while software is already booted, test that the software does not change its setting and still responds to the same subnet/node address.
3) Query and display the Neuron Configuration; analyze.
   The Location String should be as per the Table 4-1.
   The SCPTnwrkCnfg network property input should be a (manufacture time) constant value CFG_LOCAL.
4) Query and display Network Variables Configuration and binding; analyze.

4.1.1.2 Location DCU02

Same as in Section 4.1.1.1 but with location ID = DCU02

4.1.1.3 Location DCU03

Same as in Section 4.1.1.1 but with location ID = DCU03

4.1.1.4 Location DCU04

Same as in Section 4.1.1.1 but with location ID = DCU04

4.1.2 Self-Test

1) No SPI communication should result in self-test failure
   Stop SPI communication for 3 secs, the self-test request should result in self-test response with uTest1 = 1 (failed).
2) Erroneous or no Level-2 Frame communication should result in self-test failure
   Stop Level-2 Frame communication for 20 secs, the self-test request should result in self-test response with uTest1 = 1 (failed).
3) Test for uTest1=0 and uStart_Failure=0
   Set SelfTest; ensure SPI communication is stable for at least 18 seconds before sending a valid Self Test request and verifying that uTest1=0 and uStart_Failure=0.
4) Test for CRCs of Application, Boot Loader and CPLD
   In the Level 2 frame, set App_Crc32 = 0x12345678, Boot_Crc32 = 0x9ABCDEF0 and CPLD_Crc32 = 0xA9876543; ensure SPI communication is stable for at least 18 seconds before sending a valid Self Test request and verifying that the corresponding 8-character ASCII string representation of the hexadecimal 32-bits appear in App_Crc0:7, Boot_Crc0:7 & CPLD_Crc0:7.
5) Test NGA CRC and Version Over Network to be the Same as that in SPI Frame
   Verify that the 8-character ASCII string representation of NGA CRC received in the Self Test Response is the equivalent of the corresponding hexadecimal 32-bits received over the SPI. Verify that the string

representation of the NGA version received in the Software Version Response is the equivalent of the corresponding 16-bit number received in the SPI frame.

4.1.3 Time Update Function

4.1.3.1 Network Variable Parametric Tests

The time sent in these incorrectly sized packets should not get through to the SPI interface.

1) Incorrectly sized network variable, one byte too long
2) Incorrectly sized network variable, one byte too short
3) Incorrectly sized network variable, size zero

4.1.3.2 Valid MDS_Time is Sent to the SPI

1) Send a MDS_Time = 1990-01-01 00h00:00, observe if SPI receives the same time.
2) Send a MDS_Time = (local time and date), observe if SPI receives the same time.
3) Send a MDS_Time = 2100-12-31 23h45:00, observe if SPI receives the same time.
4) Send a periodic MDS_Time = (local time and date) at 5 seconds interval during 15 seconds, observe if SPI receives the same time.

4.1.3.3 Invalid MDS_Time is Not Sent to the SPI

The following test cases proceed as follows: Send one message (as per test case). Observe the MDS_Time on the SPI, which should be stamped with year=0000 (meaning the year is not specified), month=00 (meaning the month is not specified), day=00(meaning the day is not specified), and 00h00:00.

1) <u>Invalid year:</u>
   NULL year (0xFFFF), send MDS_Time = 65535-01-02 03h04:05
   Year too large, send MDS_Time = 2200-01-02 03h04:05
   Year too large, send MDS_Time = 2999-01-02 03h04:05
   Year too small, send MDS_Time = 1989-01-02 03h04:05
   Year too small, send MDS_Time = 0001-01-02 03h04:05
   Year not specified, send MDS_Time = 0000-01-02 03h04:05
2) <u>Invalid month:</u>
   Month not specified, send MDS_Time = 2007-00-02 03h04:05
   Month too large, send MDS_Time = 2007-13-02 03h04:05
   Month too large, send MDS_Time = 2007-127-02 03h04:05
   Month too large, send MDS_Time = 2007-255-02 03h04:05
   Month too large, send MDS_Time = 2007-127-02 03h04:05
3) <u>Invalid day:</u>
   Day not specified, send MDS_Time = 2007-01-00 03h04:05
   Day too large, send MDS_Time = 2007-01-32 03h04:05
   Day too large, send MDS_Time = 2007-01-127 03h04:05
   Day too large, send MDS_Time = 2007-01-255 03h04:05
4) <u>Invalid Hour:</u>
   Hour too large, send MDS_Time = 2007-01-02 24h04:05
   Hour too large, send MDS_Time = 2007-01-02 127h04:05
   Hour too large, send MDS_Time = 2007-01-02 255h04:05
5) <u>Invalid minute:</u>
   Minutes too large, send MDS_Time = 2007-01-02 03h60:05
   Minutes too large, send MDS_Time = 2007-01-02 03h127:05
   Minutes too large, send MDS_Time = 2007-01-02 03h255:05
6) <u>Invalid second:</u>
   Seconds too large, send MDS_Time = 2007-01-02 03h04:60
   Seconds too large, send MDS_Time = 2007-01-02 03h04:127
   Seconds too large, send MDS_Time = 2007-01-02 03h04:255
7) <u>Invalid day of the month:</u>
   January has 31 days, send MDS_Time = 2007-01-dd 03h04:05, where dd is day 30, 31 and 32.
   March has 31 days, send MDS_Time = 2008-03-dd 03h04:05, where dd is day 30, 31 and 32.
   April 2007 has 30 days, send MDS_Time = 2007-04-dd 03h04:05, where dd is day 30, 31 and 32.

May has 31 days, send MDS_Time = 2007-05-dd 03h04:05, where dd is day 30, 31 and 32.
June has 30 days, send MDS_Time = 2007-06-dd 03h04:05, where dd is day 30, 31 and 32.
July has 31 days, send MDS_Time = 2007-07-dd 03h04:05, where dd is day 30, 31 and 32.
August has 31 days, send MDS_Time = 2007-08-dd 03h04:05, where dd is day 30, 31 and 32.
September has 30 days, send MDS_Time = 2007-09-dd 03h04:05, where dd is day 30, 31 and 32.
October has 31 days, send MDS_Time = 2007-10-dd 03h04:05, where dd is day 30, 31 and 32.
November has 30 days, send MDS_Time = 2007-11-dd 03h04:05, where dd is day 30, 31 and 32.
December has 31 days, send MDS_Time = 2007-12-dd 03h04:05, where dd is day 30, 31 and 32.

8) Exception of leap year:
   Leap Year 2000, send MDS_Time = 2000-mm-dd 03h04:05, where mm-dd is 02-28, 02-29, 02-30, and 03-04.
   Leap year 2008, send MDS_Time = 2008-mm-dd 03h04:05, where mm-dd is 02-28, 02-29, 02-30, and 03-04.
9) Non leap year:
   Non leap year 2009, send MDS_Time = 2009-mm-dd 03h04:05, where mm-dd is 02-28, 02-29, 02-30, and 03-04.
   Non leap year 2010, send MDS_Time = 2010-mm-dd 03h04:05, where mm-dd is 02-28, 02-29, 02-30, and 03-04.
   Non leap year 2011, send MDS_Time = 2011-mm-dd 03h04:05, where mm-dd is 02-28, 02-29, 02-30, and 03-04.
10) Leap year 2012 (2012 is a leap year)
    Test with MDS_Time = 2012-mm-dd 03h04:05, where mm-dd is 02-28, 02-29, 02-30, and 03-04.
11) Non leap year 2099 (2099 is not a leap year)
    Test with MDS_Time = 2099-mm-dd 03h04:05, where mm-dd is 02-28, 02-29, 02-30, and 03-04.
12) Exception of leap year 2100 (2100 is not a leap year)
    Test with MDS_Time = 2100-mm-dd 03h04:05, where mm-dd is 02-28, 02-29, 02-30, and 03-04.

4.1.3.4 NA_Compile_Time is Sent to the SPI

Observe SPI NA_Compile_Time; verify that it is in the range 1990-01-01 thru 2100-12-31

4.1.4 DCU Status

4.1.4.1 Transmission Rate Change

1) Initial transmission rate for nvo01StatusBits, nvo01StatusBits2, and nvo01Parameter.
2) Change of the update_rate from 500 to 60,000ms (multiples of 500ms) for nvo01StatusBits, nvo01StatusBits2, and nvo01Parameter.
3) Change of the update_rate from 2,000ms to 0ms for nvo01StatusBits, nvo01StatusBits2, and nvo01Parameter.
4) Request again with same update_rate of 0ms for nvo01StatusBits, nvo01StatusBits2, and nvo01Parameter.
5) All DCU status transmission of Object #1 stops upon receiving nvi01TransfState.bit0 = 1.
6) After 60s since last reception of nvi01TransfState.bit0 = 1.

4.1.5 Network Health Monitoring

To be defined in the next revision.

4.1.6 Version Management

4.1.6.1 Normal DCU Version Request

With the SPI communication operating normally, set the Application Version = “1.2”, the Boot Loader Version = “4.5”, and CPLD Version = “7.8”.
Send “Version Request”; software should respond with its “Software Version Response”; validate the response length (4 * 43 = 172 bytes); validate all fields of the response.

4.1.6.2 Parametric Tests

1) Incorrectly sized request, one byte too long
2) Incorrectly sized request, one byte too short
3) Incorrectly sized request, size zero

4.1.6.3 DCU Version Request while No SPI Communication

Reset software through Network Command and do not set SPI data before the software version request is sent. NGA should respond with 0x00 in DCU Application, Boot Loader Application and CPLD version fields.

4.1.6.4 DCU Version Request while SPI Communication Okay but No Valid Level-2 Frame

Reset the software through Network Command between each test.

1) No Level-2 Frame at all:
   NGA should respond with 0x00 in Application, Boot Loader and CPLD version fields.
2) Application Version containing non-printable characters:
   NGA should respond with 0x00 in fields with non-printable characters.
3) Boot Loader Version containing non-printable characters:
   NGA should respond with 0x00 in fields with non-printable characters.
4) CPLD Version containing non-printable characters:
   NGA should respond with 0x00 in fields with non-printable characters.

4.1.6.5 Correctly Handle Versions that Change After Power Up

Software Version is requested, and the response is verified with following test:

1) Increment Application Version
2) Increment Boot Loader Version
3) Increment CPLD Version
4) Increment versions of all three software configuration items

4.1.7 DCU Communication via SPI Interface

1) <u>SPI Level 1 Interface Error Detection</u>
   Observe that the SPI_Outputs.SPIErr turns on/off appropriately on the next frame following the test case.

2) <u>No response on SPI (all ones)</u>
   SPIErr = 1.

3) <u>Invalid sync byte on SPI</u>
   SPIErr = 1.

4) <u>Erroneous check sum on SPI</u>
   SPIErr = 1.

5) <u>Software Ignores the SPI Bits That Should Be Ignored</u>
   i) Set to Monitoring Mode.
   ii) Set all bits that are unused in the “SPI_Frame of DCU to NGA” to value 1 for 2 seconds.
   iii) Set all value to 0 for 2 seconds.
   iv) Set all value to 1 for 200ms.
   v) Set again all value to 0.
   vi) Observe that Status Information is not sent during the test.

   Observe that the SPI.SPIErr remains 0 during the test.
   Set to Historical Mode, and download the log file. Analyze the log, it should be empty.

6) <u>Software Sends the Unused SPI Bits as Value 0</u>
   This test case is already covered by the continuous validation throughout above test.

7) SPI Level-2 Interface Error Detection
The ExtraDataID/ExtraData data pair is transmitted in ascending order of ExtraDataID from id 1 to n. One Level-2 Frame is defined as the complete sequence from 1 to n. The validation of individual field within the Level-2 Frame is performed after the Level-2 Frame is accepted. An invalid value in a field does not invalidate the entire Level-2 Frame, but only the particular field.

The software should validate and accept one Level-2 Frame only once it receives it completely. The following test cases validate missing, invalid, and extra bytes within and between Level-2 Frames.

8) Valid Level-2 Frame Synchronous over SPI
Send one complete valid Level-2 frame on the SPI, 200ms between each ExtraDataID, with valid field values. So, 86 * 200msec = 17.2 secs later, the software should have updated its software versions. Query the software versions and CRC, validating that they match the values sent in the Level-2 Frame.

9) Missing ExtraDataID 2 invalidates the Level-2 Frame
Send Level-2 frame with missing ExtraDataID number 2. Query versions, NGA should have kept the values prior to the erroneous Level-2 Frame.

10) Missing ExtraDataID 1 invalidates the Level-2 Frame
Send Level-2 frame with missing ExtraDataID number 1. Query versions, NGA should have kept the values prior to the erroneous Level-2 Frame.

11) Missing the last ExtraDataID invalidates the Level-2 Frame
Send Level-2 frame with the last ExtraDataID missing (that is number 86). Query versions; NGA should have kept the values prior to the erroneous Level-2 Frame.

12) The ExtraDataID number 1 changed for 30 invalidates the Level-2 Frame
Send complete Level-2 frame with the ExtraDataID number 1 replaced by value 30. Query versions, NGA should have kept the values prior to the erroneous Level-2 Frame.

13) The ExtraDataID number 2 changed for 86 invalidates the Level-2 Frame
Send complete Level-2 frame with the ExtraDataID number 2 replaced by value 86. Query versions, NGA should have kept the values prior to the erroneous Level-2 Frame.

14) The ExtraDataID number 86 changed for 32 invalidates the Level-2 Frame
Send complete Level-2 frame with the ExtraDataID number 86 replaced by value 32. Query versions, NGA should have kept the values prior to the erroneous Level-2 Frame.

15) An invalid ID inserted within a Level-2 Frame invalidates the Level-2 Frame
Send complete Level-2 frame with an ExtraDataID value 32 inserted between id 1 and 2. Query versions, NGA should have kept the values prior to the erroneous Level-2 Frame.

16) An invalid ID inserted between two Level-2 Frame is ignored
Send complete Level-2 frame with an ExtraDataID value 87 inserted between ID 86 of the first Level-2 Frame and ID 1 of the second one. Select field values different in the two Level-2 frames. Query software versions at the end of the second Level-2 Frame. Software should respond with field values of the second Level-2 Frame.

17) An invalid ExtraDataID does not invalidate the data frame
Send complete Level-2 frame with an ExtraDataID value 87 inserted between ID 86 of the first Level-2 Frame and ID 1 of the second one. Use the same values for all fields of the data frames for the entire duration of the two Level-2 frames, except for Emergency field. Set the Emergency=0 during the two Level-2 frames, and to Emergency=1 for the duration where the ExtraDataID value 87 is being sent. The software should send NV updates of its nvoStatus showing changes on its uEmergency_Handle_Activated field.

18) Invalid Boot Loader Version Field Values Invalidates Only the Invalid Field
Send valid Level-2 frame with different field values in each. In the second one, use an invalid character in the Boot Loader Version string, placing a non-printable character in position 2 of the string. Query software versions. Software should respond with field values of the second Level-2 Frame, except for the Boot Loader Version that should be the one from the first Level-2 Frame.

19) An Invalid Application Version Field Value Invalidates Only the Invalid Field
Send valid Level-2 frames with different fields values in the second one. In the second Level-2 Frame, use an invalid character in the Application Version string, placing a non-printable character in position 6 of the string. Query software versions. Software should respond with field values of the second Level-2 Frame, except for the Application Version that should be the one from the first Level-2 Frame.

20) Application CRC-32 Field
Test field with values 0x12345678, 0x87654321, 0x0000, and 0xffffffff. Query CRC for each.

21) Boot Loader CRC-32 Field
Test field with values 0x12345678, 0x87654321, 0x0000, and 0xffffffff. Query CRC for each.

22) CPLD CRC-32 Field
Test field with values 0x12345678, 0x87654321, 0x0000, and 0xffffffff. Query CRC for each.

23) Application Version Field, Valid Values
Test the field with values from the test vectors shown in Table 4-2. The software should accept each one. Query the software version for each test case.

Table 4-2: Version Field, Valid Test Vectors

#	Ver[0]	Ver[1]	Ver[2]	Ver[3]	Ver[4]	Ver[5]
1	0x56 (‘V’)	0x31 (‘1’)	0x32 (‘2’)	0x2E (‘.’)	0x33 (‘3’)	0x61 (‘a’)
2	0x76 (‘v’)	0x34 (‘4’)	0x35 (‘5’)	0x36 (‘6’)	0x37 (‘7’)	0x38 (‘8’)
3	0x20 (space)	0x30 (‘0’)	0x39 (‘9’)	0x38 (‘8’)	0x20 (space)	0x20 (space)
4	0x20 (space)	0x20 (space)	0x20 (space)	0x20 (space)	0x20 (space)	0x20 (space)
5	0x61 (‘a’)	0x78 (‘x’)	0x7A (‘z’)	0x41 (‘A’)	0x58 (‘X’)	0x5A (‘Z’)
6	0x28 (‘(‘)	0x29 (‘)‘)	0x2F (‘/‘)	0x5C (‘\’)	0x2B (‘+’)	0x2D (‘–’)

24) Application Version Field, Invalid Values
Test the field with values from the test vectors shown in Table 4-3. The software should not accept any one. Query the software version for each test case.

Table 4-3: Version Field, Invalid Test Vectors

#	Ver[0]	Ver[1]	Ver[2]	Ver[3]	Ver[4]	Ver[5]
1	**0x01**	0x41	0x41	0x41	0x41	0x41
2	0x41	**0x01**	0x41	0x41	0x41	0x41
3	0x41	0x41	**0x01**	0x41	0x41	0x41
4	0x41	0x41	0x41	**0x01**	0x41	0x41
5	0x41	0x41	0x41	0x41	**0x01**	0x41
6	0x41	0x41	0x41	0x41	0x41	**0x01**
7	**0x02**	0x41	0x41	0x41	0x41	0x41
8	**0x1F**	0x41	0x41	0x41	0x41	0x41
9	**0x7F**	0x41	0x41	0x41	0x41	0x41
10	**0x80**	0x41	0x41	0x41	0x41	0x41
11	**0x81**	0x41	0x41	0x41	0x41	0x41
12	**0xFF**	0x41	0x41	0x41	0x41	0x41
13	**0x00**	0x41	0x41	0x41	0x41	0x41

25) Boot Loader Version Field, Valid Values
Test the field with values from the test vectors shown in Table 4-2. The software should accept each one. Query the software version for each test case.

26) Boot Loader Version Field, Invalid Values
Test the field with values from the test vectors shown in Table 4-3. The software should not accept any one. Query the software version for each test case.

27) CPLD Version Field, Valid Values
Test the field with values from the test vectors shown in Table 4-2. The software should accept each one. Query the software version for each test case.

28) CPLD Version Field, Invalid Values
Test the field with values from the test vectors shown in Table 4-3. The software should not accept any one. Query the software version for each test case.

29) Neuron Application Information Verification
Verify the following Level-2 SPI communication information related to the Neuron Application:
a) NA_Crc32 is the same as that entered by the user.
b) NA_VerMaj and NA_VerMin are the same as that entered by the user.
c) NA_CompileTime (year, month and day) are the same as entered by the user.
d) NA_StringDescription is “NEURON APP”.

4.1.8 SCI Update

4.1.8.1 SCI Image Transfer from the Network

To be defined in the next revision.

4.1.8.2 SCI Image Transfer to DCU Application over SPI

To be defined in the next revision.

4.1.9 LonMark® Interoperability

1) SCPTnwrkCnfg is a manufacture-time constant set to CFG_LOCAL
Query the configuration, analyze.

2) nvi00ObjRequest.object_request = RQ_NORMAL
Test with nvi00ObjRequest.object_id =0, 1, 2, and 3.
NGA returns specified functional block to normal operation.

3) nvi00ObjRequest.object_request = RQ_UPDATE_STATUS
Test with nvi00ObjRequest.object_id =0, 1, 2, and 3.
NGA updates the SNVT_obj_status (if the Node Object is specified then inclusive-OR of the status bits for all functional blocks are applied).

4) nvi00ObjRequest.object_request = RQ_REPORT_MASK
Test with nvi00ObjRequest.object_id =0, 1, 2, and 3.
NGA sets only used bits status to 1 (if the Node Object is specified then inclusive-OR of supported status bits for all functional blocks are applied).

5) nvi00ObjRequest.object_request = RQ_CLEAR_STATUS
Test with nvi00ObjRequest.object_id =0, 1, 2, and 3.
NGA clears the status bits of the specified functional block (if the Node Object is specified then all functional blocks are applied).

6) Invalid object_id and object_request
NGA returns invalid_id with value 1 for invalid object_id, and it returns invalid_request with value 1.

5 TEST PROCEDURE

5.1 Special Requirements

The following tables list the equipment, the software and the special files used in the testing:

Table 5-1: Hardware Required

Hardware	Version	P/N	Rev.
Computer, minimum PENTIUM, with Windows XP (for PTU simulator application and PTU Application)	--	--	--
Complete Door System: 1 Door Control Unit (DCU) 1 Door Control Panel 2 Door Operators 2 Solenoid Locks 2 Motors	--	--	--
PTU cable DB9-PTU PLUG	--	--	--
Variable Power Supply (Voltage source min. 10A, 0-50V)	--	--	--
DCU Test Fixture	--	--	--
Banana cables	--	--	--
Multimeter	--	--	--

Table 5-2: Software Required

Software	Version	P/N	Rev.
Neuron Gateway Application	0.0.1	2011L0052	0
DCU Application	0.01	2011L0051	0
Boot Loader	0.01	2011L0053	0
PTU Application	0.0.1	2011L0054	0
PTU Simulator	0.0.1	-	0
SPI Validation Tool software	-	-	0
Network Validation Tool software	-	-	0
NodeUtil V1.82	-	-	0
OpenLDV™ Installer V3.0	1.0	2004L0036	0
PCLTA-10/20 Driver for Windows 2000, Echelon P/N 153-0027-01 Rev. B	2.01B	2002L0046	0

5.2 Procedure Steps

5.2.1 Test Log

The result of each test case log should be recorded in the last column,"P/F", of sections 5.4.1 to 5.4.9. Letter "P" or "F" will indicate Pass or Fail to the test.

Record the test results onto a hardcopy (print) of the Appendix A of this document. Sign and date the document. Copies of Appendix A may be done if test is segmented. Create a new revision of Integration Test Report 201010749 and append this Integration Test Procedure with the results filled in. Record the test anomalies that need investigation, if any, as System Problem Reports (SPR) in the ClearQuest database.

5.2.2 Test Setup

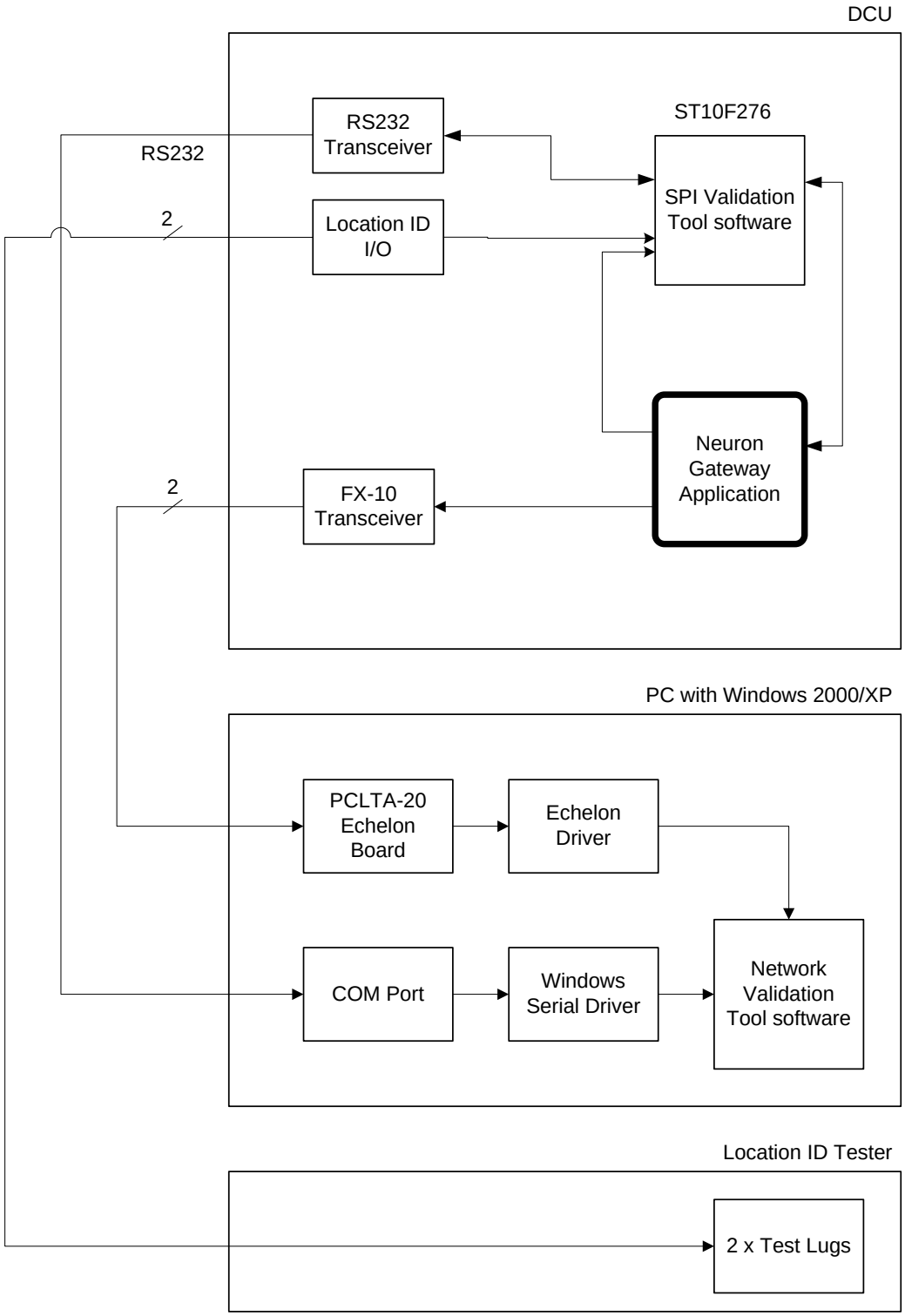

Figure 5-1: Test Setup

5.2.2.1 PTU Application Installation

The PTU software comes with an automated Windows-based installation program called SETUP. Don't try copying the files manually on your hard disk; instead please follow these steps to install the software:

1. Extract from the PTU installation zip file (as listed in Table 5-2) the setup.exe file.
2. Run SETUP.EXE.
3. The setup program will then begin to copy and decompress files on the target drive. You will be prompted when it is necessary to insert subsequent diskettes.
4. The setup program will prompt you to terminate any application that might lock system resources or files. A good practice would be to stop any other application before running setup.
5. The next step will be to select a target directory for the program. It is recommended that the default location be used if possible.
6. Once the target directory has been selected and approved, the setup program initiates the installation procedure. Once the process is over, the program will inform you if the installation was successful or not.

5.2.2.2 DCU Boot Loader Programming

To upload the Boot Loader Application in the DCU, you have to execute the following steps:

1. Turn DCU power OFF.
2. Connect PTU computer and DCU with PTU cable.
3. On the DCU, install the jumper called "Boot Strap" mode.
4. Use PTU Application.
5. Select Tools -> Load Flash -> Open Flash File to open the Boot Loader Application (H86 format).
6. Select DCU Boot Loader (as listed in Table 5-2) and click OK.
7. Select Tools -> Load Flash -> Send Flash File to start to upload the Boot Loader Application. Follow the uploading prompt of the PTU.
8. After Boot Loader Application process is finished, verify if the uploading successful information is presented. If the uploading is successful, follow the prompt to remove the PTU cable and reset the DCU and remove the "Boot Strap" and the "Reset" jumper. If the uploading is failed, redo steps 6 and 7.
9. The DCU is programmed with the Boot Loader Application version.

5.2.2.3 SPI Validation Tool Software Programming

To upload the SPI Validation Tool software in the DCU, execute the following steps:

1. Connect PTU computer and DCU with PTU cable.
2. Turn DCU power ON.
3. Use PTU Application.
4. Select Tools -> DCU Update -> Open DCU App. File.
5. Select the file (as listed in Table 5-2) and click OK
6. After the file verification, click OK
7. Select Tools -> DCU Update -> Update.
8. Click YES.
9. When the upload is finished successfully, click OK on the prompt window.

5.2.2.4 Neuron Gateway Application Upgrade through the network

1. Make sure the PTU cable network connector is connected to the Echelon interface card of the PC.
2. Start the NodeUtil.exe program. The following menu will appear.

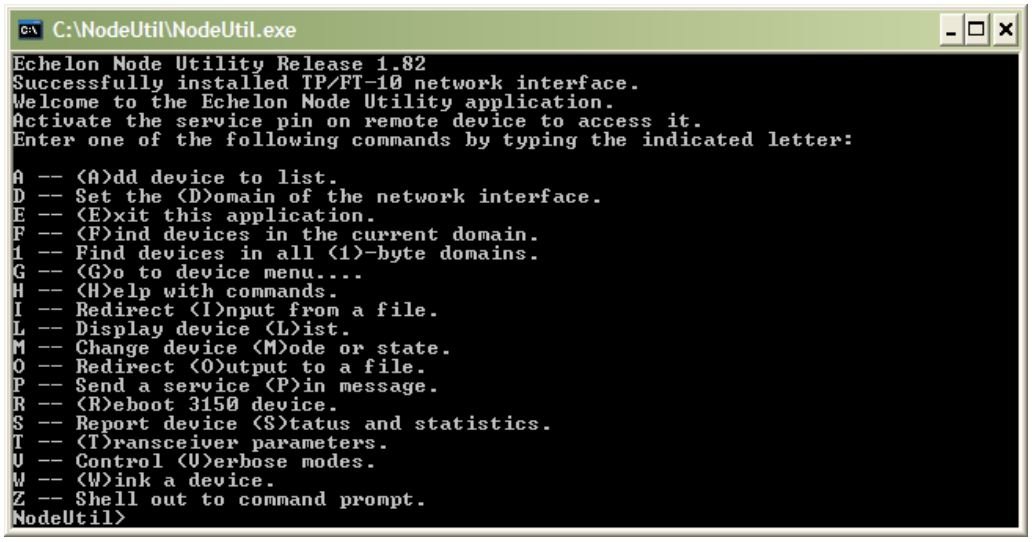

Figure 5-2: NodeUtil Main Menu

3. Send the Service Pin from the DCU by shorting pins 1 and 2 of test connector TC-4 with a metallic object. NodeUtil will receive a message from the DCU.
4. Type “G” to go to device menu. Type 1 and press [Enter] to chose node 1 (node 1 is the DCU that just sent the service pin). The following menu will appear.

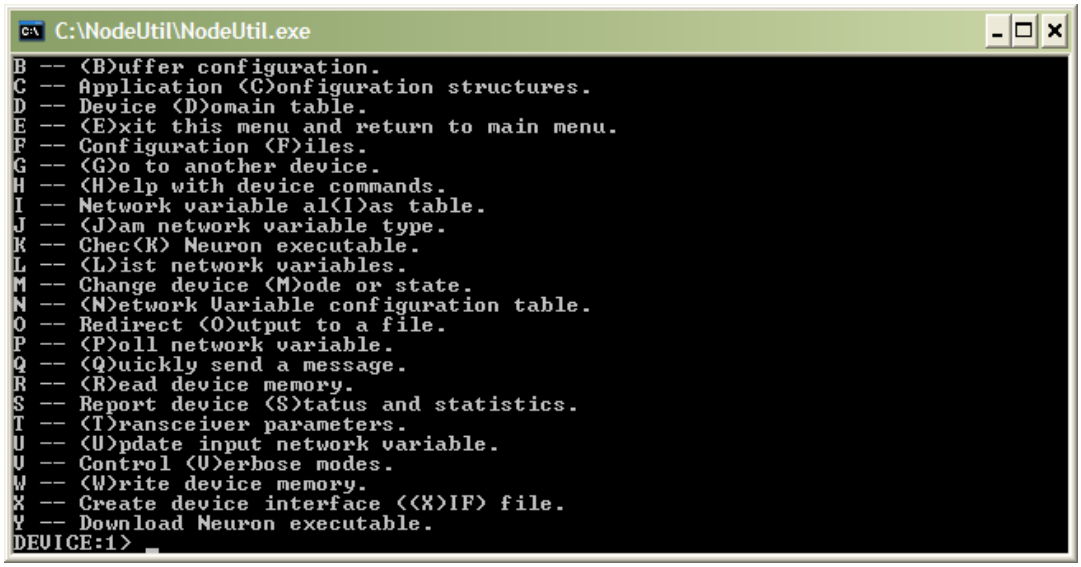

Figure 5-3: NodeUtil Device Menu

5. Type “Y” to Download Neuron Executable. Type the name of the file to program R188_nga.nxe extracted from the NGA package (as listed in Table 5-2) and press [Enter]. Note: if the file is not located in the same directory as NodeUtil, type the full path (ex.: c:\folder\sub-folder\xxx.nxe).
6. When the file is successfully downloaded, press “Y” to continue.
7. Press “E” twice to exit NodeUtil.

5.2.2.5 OpenLDV Installer Software Installation

To install the OpenLDV Installer software onto the PC, execute the following steps:

1. Extract from the OpenLDV Software installation zip file (as listed in Table 5-2) the OpenLDV.exe file that is the installation software for OpenLDV driver.
2. Run OpenLDV.exe
3. The setup program will then begin to copy and decompress files on the target drive. You will be prompted when it is necessary to click on “next” button to proceed to installation.
4. The setup program will ask the agreement on the software license. Choose “I Accept the terms in the license agreement” and click on “next”.
5. The installation of the OpenLDV Echelon driver will begin. Once completed, the installation program will terminate automatically.

5.2.2.6 NodeUtil Software Installation

The NodeUtil Software shall be executed /installed only after the OpenLDV driver has been already installed. To install the NodeUtil software onto the PC, execute the following steps:

1. Extract from the nodeUtil Software installation zip file (as listed in Table 5-2) the NodeUtil.exe file that is the executable software for NodeUtil.

5.2.2.7 Other Information about the Test

In this test procedure, each test section is independent. Within a section, the initial setup is described in the “Special Procedural Requirements” and all test cases must be executed sequentially.

All LEDs referred in this test procedure are on the DCU.

When it is required to uses the PTU Application, it is required to log into the DCU, establish a session, and open a new project window in the “File” menu. When you close the PTU Application, it is not required to save the project otherwise mentioned in the procedure.

Do all tests with the power supply set to 37.5 Volts DC +/- 5V. Verify with the Voltmeter the supply voltage.

Set the DCU Location DCU R1 as seen in Figure 5-4 and Table 5-3, unless otherwise stated in a test.

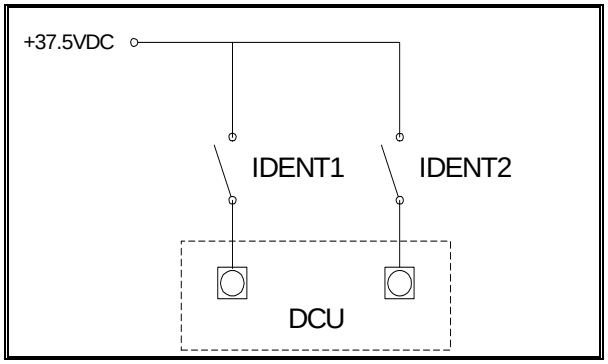

Figure 5-4: Electrical Connections of the Location ID Switches

Table 5-3: Location ID and IDENT Switches

DCU Location	IDENT1	IDENT2
DCU R1	Off	Off
DCU R2	On	Off
DCU L1	On	On
DCU L2	Off	On

5.3 Procedure

In the following test procedure, the used software configuration items are: Network Validation Tool (NVT), and NodeUtil.

The Network Validation Tool and NodeUtil share one LonWorks™network interface card (PCLTA-20). There is only one application, either Network Validation Tool or NodeUtil, running and communicating to the DCU at any given time.

The software communicates to the DCU via the LonWorks™ network interface card. The driver must be installed properly.

To use software, follow the test case instructions.

5.3.1 TC3.1 Neuron Node Self-Installation

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.1.1 DCU R1	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
2	TC3.1.2 DCU R2	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
3	TC3.1.3 DCU L1	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
4	TC3.1.4 DCU L2	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

5.3.2 TC3.2 Self-Test

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.2.1 Power on Self Tests	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

5.3.3 TC3.3 Time Update with Network

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.3.1 NV Parametric Tests	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
2	TC3.3.2 Valid MDS_Time is Sent to the SPI	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
3	TC3.3.3 Invalid MDS_Time is Not Sent to the SPI	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
4	TC3.3.4 Invalid MDS_Time is Not Used for Time Stamping Events (Historical Log file)	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
5	TC3.3.5 Compile_Time is Sent to the SPI	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
6	TC3.3.6 Invalid Disp_Time Does Not Affect Compile_Time sent to the SPI	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
7	TC3.3.7 The ReceivedTime of the SPI is set for 200 ms upon reception of a PresentTime	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

5.3.4 TC3.4 DCU Status

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.4.1 Transmission Rate Change	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

5.3.5 TC3.5 Network Health Monitoring

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.5.1 Verify the Network Health Monitoring when there is sudden loss of communication.	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

5.3.6 TC3.6 Version Management

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Note: Network Validation Tool needs to be restarted before executing this test case.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.6.1 Normal DCU Version Request	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
2	TC3.6.2 Parametric Tests	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
3	TC3.6.3 DCU Version Request with an Invalid usFunctionID	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
4	TC3.6.4 DCU Version Request While No SPI Communication	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
5	TC3.6.5 DCU Version Request While SPI Communication Okay but No Valid Level-2 Frame	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
6	TC3.6.6 Correctly Handle Versions that Change After Power Up	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

5.3.7 TC3.7 SCI Update

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.7.1 Update DCU Application	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

5.3.8 TC3.8 DCU Communication via SPI interface

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Note: Network Validation Tool needs to be restarted before executing this test case.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.8.1 SPI Level 1 Interface Error Detection	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
2	TC3.8.2 Software Ignores the SPI Bits That Should Be Ignored	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
3	TC3.8.3 Software Ignores the SPI Bits That Are Not Supported	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
4	TC3.8.4 Software Sends the Unused SPI Bits as Value 0	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
5	TC3.8.5 SPI Level-2 Interface Error Detection	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
6	TC3.8.6 Application CRC Field	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
7	TC3.8.7 Boot Loader CRC Field	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
8	TC3.8.8 Application Version Field, Valid Values	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
9	TC3.8.9 Application Version Field, Invalid Values	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
10	TC3.8.10 Boot Loader Version Field, Valid Values	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
11	TC3.8.11 Boot Loader Version Field, Invalid Values	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
12	TC3.8.12 Location ID Field	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

5.3.9 TC3.9 LonMark® Interoperability

Special Procedural Requirements: *The tester shall follow the steps asked by the automatic test software and execute the actions requested on the switches of the test fixture.*

The results of the test should be recorded on the corresponding section of the test sheet.

Measurements: *Self-explanatory.*

#	Test Cases	Input Specifications	Output Specifications	P/F
1	TC3.9.1 SCPTnwrkCnfg is a manufacture-time constant set to CFG_LOCAL	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
2	TC3.9.2 nvi00ObjRequest.object_request = RQ_NORMAL	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
3	object_request = RQ_UPDATE_STATUS	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
4	nvi00ObjRequest.object_request = RQ_CLEAR_STATUS	Launch NVT and initiate test case.	NVT should indicate that the test passed.	
5	nvi00ObjRequest.object_request = RQ_CLEAR_STATUS	Launch NVT and initiate test case.	NVT should indicate that the test passed.	

APPENDIX A: TEST REPORT SHEET

(Page 1 of 3)

Item Identification

Item Tested:	Neuron Gateway Application
Item P/N:	2011L0052
Item Version & compilation date:	

Tester Identification

Tester(s):	
Date & time when the tests started:	
Date & time when the tests finished:	
Physical Location of Test Execution:	

Equipment Used

Description	Part Number	Rev.	Serial Number	Calibration Due Date
Multimeter				
Computer (Network Validation Tool, NodeUtil, PTU Application)				N/A
DCU Test Fixture				
DCU Assy				N/A

Software Description	Version	Date
DCU Application		
Neuron Gateway Application		
Boot Loader		
PTU Application		
PTU Simulator		
SPI Validation Tool software		
Network Validation Tool software		
NodeUtil V1.82		
OpenLDV™ Installer V3.0		
PCLTA-10/20 Driver for Windows 2000, Echelon P/N 153-0027-01 Rev. B		

Test Report Sheet (Page 2 of 3)

Comment:

Approvals

Tester:		Title:		Date:	
Witness:		Title:		Date:	

Make copies of this page if required

Test Report Sheet (Page 3 of 3)

Summary of Anomaly:

SPR #	Description

Make copies of this page if required